

Q1. Complete Query Processing Pipeline

The question asks for pseudocode covering the complete pipeline from **SQL input** → **parsing** → **optimization** → **execution** → **result output**.

Pseudocode

BEGIN

INPUT SQL_Query

// Step 1: Parse the SQL query

Tokens ← Lexical_Analysis(SQL_Query)

ParseTree ← Syntax_Analysis(Tokens)

// Step 2: Check the query

Validate(ParseTree)

IF Query is Invalid THEN

 DISPLAY "Invalid SQL Query"

 STOP

END IF

// Step 3: Create query representation

QueryTree ← Create_Query_Tree(ParseTree)

// Step 4: Optimize the query

OptimizedPlan ← Optimize(QueryTree)

// Step 5: Execute the optimized plan

Result ← Execute(OptimizedPlan)

// Step 6: Display result

DISPLAY Result

END

Simple Flow

SQL Query



Lexical Analysis



Syntax Analysis



Validation



Query Tree



Query Optimization



Optimized Execution Plan



Query Execution



Result Output

Explanation

1. **Input:** Accept the SQL query.
2. **Parsing:** Convert the SQL query into tokens and a parse tree.
3. **Validation:** Check whether the query is valid.
4. **Query Tree:** Represent the query as a query tree.
5. **Optimization:** Find an efficient execution plan.
6. **Execution:** Execute the optimized plan.
7. **Output:** Display the query result.

Conclusion

The query processing pipeline converts an SQL query into an **optimized execution plan** and executes it to produce the final result.

Q2. Cost-Based Query Optimization

The question asks for a **pseudocode algorithm for cost-based query optimization** that estimates and compares **I/O costs for different join orderings**.

Pseudocode

BEGIN

INPUT: Relations and Join Conditions

Generate all possible join orderings

FOR each Join Ordering DO

Cost $\leftarrow$ 0

Intermediate_Result $\leftarrow$ First Relation

FOR each next Relation in Join Ordering DO

Estimate number of pages of

Intermediate_Result

Estimate number of pages of

next Relation

Join_Cost $\leftarrow$ Calculate_IO_Cost(

Intermediate_Result,

next Relation

)

Cost $\leftarrow$ Cost + Join_Cost

Intermediate_Result $\leftarrow$

```
    Estimate_Join_Result(  
        Intermediate_Result,  
        next Relation  
    )
```

END FOR

Store Join Ordering and Cost

END FOR

Choose the Join Ordering
with the minimum Cost

OUTPUT Optimized Join Ordering

END

Simple Steps

1. **Input** the relations and join conditions.
2. Generate different possible **join orderings**.
3. Calculate the **I/O cost** for each ordering.
4. Estimate the size of each intermediate result.
5. Add the costs of all joins.
6. Compare the total costs.
7. Select the join ordering with the **lowest I/O cost**.

Example

Suppose there are three relations:

R1, R2, R3

Possible join orders:

$(R1 \bowtie R2) \bowtie R3$

$(R1 \bowtie R3) \bowtie R2$

$(R2 \bowtie R3) \bowtie R1$

The optimizer calculates the I/O cost for each:

$(R1 \bowtie R2) \bowtie R3 \rightarrow 5000 \text{ I/O}$

$(R1 \bowtie R3) \bowtie R2 \rightarrow 3000 \text{ I/O}$

$(R2 \bowtie R3) \bowtie R1 \rightarrow 4500 \text{ I/O}$

Therefore:

Minimum Cost = 3000 I/O

Best Join Order = $(R1 \bowtie R3) \bowtie R2$

Conclusion

Cost-based optimization compares different execution plans and selects the **join ordering with the lowest estimated I/O cost**, improving query performance.

Q3. Nested Loop Join Algorithm

The question asks for **pseudocode for the Nested Loop Join algorithm** and its time complexity in terms of **buffer pages and tuple counts**.

Pseudocode

BEGIN

INPUT: Relation R, Relation S

FOR each tuple r in R DO

 FOR each tuple s in S DO

 IF Join_Condition(r, s) = TRUE THEN

 OUTPUT (r, s)

 END IF

 END FOR

END FOR

END

Simple Explanation

- Take one tuple from **R**.
- Compare it with **every tuple in S**.
- If the join condition is satisfied, output the matching tuples.
- Repeat until all tuples of R are processed.

Example

Suppose:

$R = \{R1, R2\}$

$S = \{S1, S2, S3\}$

Comparisons are:

$R1 \rightarrow S1, S2, S3$

$R2 \rightarrow S1, S2, S3$

Total comparisons:

$$2 \times 3 = 6$$

Time Complexity

If:

- $T(R)$ = number of tuples in R
- $T(S)$ = number of tuples in S

Then the tuple comparison cost is:

$$\text{Time Complexity} = O(T(R) \times T(S))$$

For the **page-based block nested-loop join**, if:

- $B(R)$ = number of pages in R
- $B(S)$ = number of pages in S
- M = number of available buffer pages

The approximate I/O cost is:

$$B(R) + \text{ceil}(B(R) / (M - 2)) \times B(S)$$

Conclusion

Nested Loop Join is simple and easy to implement, but it can be expensive for large relations because it may compare a large number of tuples. Using multiple buffer pages in a block nested-loop join can reduce the I/O cost.

Q4. Deadlock Detection Using Wait-For Graph

The question asks for a pseudocode algorithm to **detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.**

Pseudocode

BEGIN

INPUT: Set of Transactions and Lock Requests

Create an empty Wait-For Graph (WFG)

FOR each transaction T1 DO

 FOR each lock request of T1 DO

 IF the requested lock is held by transaction T2 THEN

 Add edge $T1 \rightarrow T2$ to WFG

 END IF

 END FOR

END FOR

// Check for a cycle

IF WFG contains a cycle THEN

 DISPLAY "Deadlock Detected"

 DISPLAY the transactions involved in the cycle

```
ELSE
    DISPLAY "No Deadlock"
END IF
```

END

Example

Suppose:

T1 is waiting for T2

T2 is waiting for T3

T3 is waiting for T1

The Wait-For Graph becomes:

T1 → T2

↑ ↓

T3 ←——

There is a cycle:

T1 → T2 → T3 → T1

Therefore:

Deadlock Detected

Steps

1. Create a **Wait-For Graph**.
2. Add each transaction as a node.
3. If T1 is waiting for a lock held by T2, add an edge **T1 → T2**.
4. Check the graph for a cycle.
5. **Cycle present → Deadlock exists.**
6. **No cycle → No deadlock.**

Conclusion

A Wait-For Graph provides a simple way to detect deadlocks. A **cycle in the graph indicates that transactions are waiting for each other**, so a deadlock has occurred.

Q5. Basic Two-Phase Locking (2PL)

The question asks for **pseudocode for the Basic Two-Phase Locking protocol**, including **lock request, grant, wait, and release phases**.

Pseudocode

BEGIN

INPUT: Transaction T and Data Item D

// Lock Request

REQUEST_LOCK(T, D, Lock_Type)

IF D is not locked THEN

 GRANT Lock_Type to T

ELSE IF Lock_Type is compatible with

 the existing lock THEN

 GRANT Lock_Type to T

ELSE

 ADD T to Waiting_List

 WAIT until the lock becomes available

END IF

// Transaction Operations

PERFORM Read/Write operations

// Lock Release

WHEN transaction T completes DO

 RELEASE all locks held by T

FOR each waiting transaction DO

 IF its requested lock can be granted THEN

 GRANT the lock

```

    REMOVE transaction from Waiting_List
  END IF
END FOR

```

```

END WHEN
END

```

Two Phases of 2PL

1. Growing Phase

Transaction can acquire locks

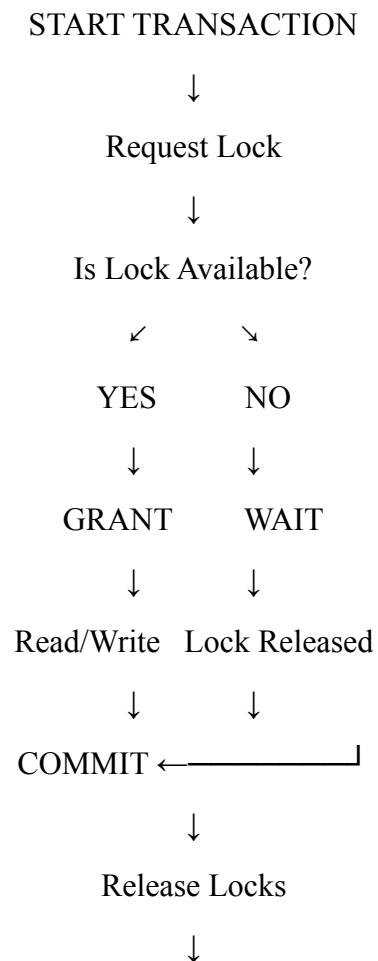
Transaction cannot release locks

2. Shrinking Phase

Transaction can release locks

Transaction cannot acquire new locks

Simple Flow



END

Example

T1 requests X(A)



A is available



Grant X(A)



T1 writes A



T1 COMMIT



Release X(A)

If another transaction requests the same lock while T1 is using it:

T2 requests X(A)



A is locked by T1



T2 WAIT



T1 COMMIT



T1 releases A



Grant X(A) to T2

Conclusion

Basic **2PL** divides transaction execution into a **growing phase**, where locks are acquired, and a **shrinking phase**, where locks are released. This helps maintain **serializability and consistency** in concurrent transactions.

Q6. Timestamp-Based Concurrency Control Using Thomas' Write Rule

The question asks for **pseudocode for timestamp-based concurrency control using Thomas' Write Rule** to handle conflicting read/write operations.

Pseudocode

BEGIN

INPUT: Transaction T and Data Item X

Assign timestamp $TS(T)$ to transaction T

FOR each operation of T DO

IF operation = READ(X) THEN

IF $TS(T) < WTS(X)$ THEN

ABORT T

ELSE

PERFORM READ(X)

$RTS(X) = \max(RTS(X), TS(T))$

END IF

ELSE IF operation = WRITE(X) THEN

IF $TS(T) < RTS(X)$ THEN

ABORT T

ELSE IF $TS(T) < WTS(X)$ THEN

// Thomas' Write Rule

IGNORE the WRITE operation

```
ELSE
    PERFORM WRITE(X)
    WTS(X) = TS(T)
END IF
```

```
END IF
```

```
END FOR
```

```
COMMIT T
```

```
END
```

Meaning of Timestamps

- **TS(T)** → Timestamp of transaction T.
- **RTS(X)** → Largest timestamp of a transaction that has successfully read X.
- **WTS(X)** → Largest timestamp of a transaction that has successfully written X.

Thomas' Write Rule

The important condition is:

IF $TS(T) < WTS(X)$

THEN

IGNORE WRITE(X)

This means the write is **obsolete**, so instead of aborting the transaction, the write operation is simply ignored.

Simple Example

Suppose:

$TS(T1) = 10$

$WTS(X) = 20$

T1 wants to execute:

WRITE(X)

Since:

$TS(T1) < WTS(X)$

$10 < 20$

Thomas' Write Rule is applied:

IGNORE WRITE(X)

T1 can continue without aborting because the write is already outdated.

Important Cases

Condition	Action
$TS(T) < RTS(X)$ for WRITE	Abort T
$TS(T) < WTS(X)$ for WRITE	Ignore WRITE
Otherwise for WRITE	Perform WRITE
$TS(T) < WTS(X)$ for READ	Abort T
Otherwise for READ	Perform READ

Conclusion

Thomas' Write Rule improves timestamp-based concurrency control by **ignoring obsolete write operations instead of unnecessarily aborting the transaction**. This can improve concurrency and reduce transaction rollbacks.

Q7. ARIES Recovery – Pseudocode

Algorithm: ARIES_RECOVERY

Input: Log records, Transaction Table, Dirty Page Table

Output: Recovered and consistent database

BEGIN ARIES_RECOVERY

/* PHASE 1: ANALYSIS */

Read the log from the last checkpoint

Initialize Transaction Table

Initialize Dirty Page Table

FOR each log record DO

IF record is BEGIN TRANSACTION THEN

 Add transaction to Transaction Table

ELSE IF record is UPDATE THEN

 Update transaction information

IF page is not in Dirty Page Table THEN

 Add page to Dirty Page Table

END IF

ELSE IF record is COMMIT THEN

 Mark transaction as COMMITTED

ELSE IF record is ABORT THEN

 Mark transaction as ABORTING

ELSE IF record is END THEN

 Remove transaction from Transaction Table

END IF

END FOR

Identify:

 Winner transactions = committed transactions

 Loser transactions = incomplete transactions

/* PHASE 2: REDO */

Find the earliest LSN from the Dirty Page Table

FOR each log record from the earliest LSN DO

IF record is REDOABLE THEN

REDO the operation on the database page

END IF

END FOR

/* PHASE 3: UNDO */

FOR each loser transaction DO

Start from its last log record

WHILE undo records exist DO

Undo the operation

Write a Compensation Log Record (CLR)

Move to the previous log record

END WHILE

Write an END record

END FOR

Flush required log records to stable storage

Return recovered database

END ARIES_RECOVERY

Explanation

1. **Analysis:** Reads the log from the last checkpoint and identifies the **winner and loser transactions** and the **dirty pages**.
2. **Redo:** Repeats the necessary logged operations to restore the database to the state just before the crash.
3. **Undo:** Reverses the operations of incomplete/loser transactions and records the undo actions using **Compensation Log Records (CLRs)**.

Flow:

Crash → Analysis → Identify Transactions & Dirty Pages → Redo → Undo Loser Transactions → Consistent Database

Q8. Shadow Paging Recovery – Pseudocode

The question asks you to design pseudocode for **Shadow Paging recovery**, maintaining **current and shadow page tables** and rolling back on failure.

Pseudocode

Algorithm: SHADOW_PAGING_RECOVERY

Input: Database pages and page table

Output: Recovered and consistent database

BEGIN SHADOW_PAGING_RECOVERY

/* INITIALIZATION */

Create Shadow_Page_Table

Create Current_Page_Table

Copy Current_Page_Table to Shadow_Page_Table

/* DURING TRANSACTION */

FOR each page that needs modification DO

Create a new copy of the page

Modify the new page

Update Current_Page_Table
to point to the new page
END FOR

/* COMMIT */

IF transaction completes successfully THEN

Flush modified pages to disk

Update the database root pointer
to point to Current_Page_Table

Commit transaction

END IF

/* FAILURE / CRASH */

IF system failure occurs THEN

Discard Current_Page_Table

Restore database root pointer
to Shadow_Page_Table

Restore database to its
previous consistent state

END IF

END SHADOW_PAGING_RECOVERY

Explanation

1. **Shadow Page Table:** Keeps the original page addresses and remains unchanged during the transaction.
2. **Current Page Table:** Contains the updated page addresses after modifications.
3. **Page Modification:** Modified pages are written to new locations instead of overwriting the original pages.
4. **Commit:** After successful completion, the root pointer is changed to the **Current Page Table**.
5. **Failure:** If a crash occurs before commit, the Current Page Table is discarded and the **Shadow Page Table** is used to restore the previous consistent state.

Flow

Create Shadow & Current Tables → Modify Pages → Update Current Table → Commit → Update Root Pointer

If Failure:

Failure → Discard Current Table → Restore Shadow Table → Consistent Database

Q9. Immediate Update Recovery – Pseudocode

The question asks for pseudocode for **Immediate Update Recovery using UNDO/REDO log processing after a system crash**.

Pseudocode

Algorithm: IMMEDIATE_UPDATE_RECOVERY

Input: Database, transaction log

Output: Consistent database after system crash

BEGIN IMMEDIATE_UPDATE_RECOVERY

/* BEFORE CRASH */

FOR each transaction T DO

FOR each update operation of T DO

Write the log record:

<T, Data_Item, Old_Value, New_Value>

Write the log record to stable storage

Update the database with New_Value

END FOR

END FOR

/* AFTER SYSTEM CRASH */

Read the transaction log

Identify:

Committed transactions = WINNERS

Uncommitted transactions = LOSERS

/* REDO PHASE */

FOR each WINNER transaction DO

FOR each update record of the transaction DO

Write New_Value to the database

END FOR

END FOR

/* UNDO PHASE */

FOR each LOSER transaction DO

FOR each update record of the transaction
in reverse order DO

Write Old_Value back to the database

END FOR

END FOR

Write END records for recovered transactions

Flush all updated pages to stable storage

Return consistent database

END IMMEDIATE_UPDATE_RECOVERY

Explanation

1. **Log the update:** Before changing the database, store the **old value and new value** in the log.
2. **Immediate update:** The new value can be written to the database before the transaction commits.
3. **After crash:** Identify **committed (winner)** and **uncommitted (loser)** transactions.
4. **REDO:** Reapply the **new values** of committed transactions.
5. **UNDO:** Restore the **old values** of uncommitted transactions in reverse order.

Example

Suppose the log contains:

<T1, A, 100, 150>

<T2, B, 200, 250>

COMMIT T1

CRASH

- **T1** is a winner → **REDO**: A = 150
- **T2** is a loser → **UNDO**: B = 200

Therefore, the database is restored to a **consistent state** after the crash.

Flow:

Update → Write UNDO/REDO Log → Crash → Identify Winners & Losers → REDO Winners → UNDO Losers → Consistent Database.

Q10. Database Connectivity Manager – Pseudocode

The question asks for a **database connectivity manager** that handles **connection pooling, query execution, and exception rollback**.

Pseudocode

Algorithm: DATABASE_CONNECTIVITY_MANAGER

Input: Database configuration and SQL query

Output: Query result or error message

BEGIN DATABASE_CONNECTIVITY_MANAGER

 Initialize Connection_Pool

 /* CREATE CONNECTION POOL */

 IF Connection_Pool is empty THEN

 Create required database connections

 Add connections to Connection_Pool

 END IF

 /* GET CONNECTION */

 connection ← Get an available connection

 from Connection_Pool

 IF connection is NULL THEN

 Display "No connection available"

 RETURN

END IF

TRY

/* START TRANSACTION */

BEGIN TRANSACTION

/* EXECUTE QUERY */

query $\leftarrow$ Receive SQL query

result $\leftarrow$ Execute query using connection

IF query execution is successful THEN

COMMIT TRANSACTION

Return connection to Connection_Pool

Display result

ELSE

ROLLBACK TRANSACTION

Return connection to Connection_Pool

Display "Query execution failed"

END IF

CATCH Exception

```
/* EXCEPTION HANDLING */
```

```
ROLLBACK TRANSACTION
```

```
Return connection to Connection_Pool
```

```
Display "Database Error"
```

```
END TRY
```

```
/* CLOSE CONNECTION POOL */
```

```
IF application is shutting down THEN
```

```
    Close all connections in Connection_Pool
```

```
END IF
```

```
END DATABASE_CONNECTIVITY_MANAGER
```

Explanation

1. **Connection Pooling:** A pool of reusable database connections is maintained instead of creating a new connection for every query.
2. **Get Connection:** An available connection is taken from the pool.
3. **Query Execution:** The SQL query is executed within a transaction.
4. **Commit:** If execution is successful, the transaction is committed.
5. **Rollback:** If an error or exception occurs, the transaction is rolled back to maintain database consistency.
6. **Return Connection:** After execution, the connection is returned to the connection pool.

Flow

Create Connection Pool → Get Connection → Begin Transaction → Execute Query → Commit → Return Connection

If Exception:

Exception → Rollback → Return Connection → Display Error